## Explanation of the PennySortingParallel.java Program

The following provides a high-level overview of the three parallel sorting approaches used in the PennySortingParallel.java program. Each implementation sorts an identical copy of the same 1,000,000-element integer array, enabling students to compare how different Java parallel programming techniques solve the same problem. Because this discussion is intended for an introductory CS1 course, it focuses on the key concepts and programming models rather than the low-level implementation details of the underlying algorithms and runtime system.

## 1. Arrays.parallelSort() — Built-in data parallelism

Arrays.parallelSort(arr1);

**What it does**

- Divides the array into smaller subarrays.
- Sorts eligible subarrays in parallel.
- Merges the sorted subarrays into one sorted array.
- May use sequential sorting when a subarray is below an internal size threshold.

**How it works internally**

- Uses Java's Fork/Join framework and a common Fork/Join pool.
- Implements a parallel sort-merge approach.
- At the smallest level, it uses the appropriate Arrays.sort() method to sort individual subarrays.
- For an int[], the corresponding sequential Arrays.sort() implementation uses Dual-Pivot Quicksort.

**Key idea: Data parallelism**

The same sorting operation is performed concurrently on different portions of the array.

**Why it is useful**

- Provided and optimized by the Java runtime.
- Requires no manual creation or management of threads.
- Offers a simple way to sort sufficiently large arrays in parallel.
- Automatically determines how to divide the work.

**Important nuance**

Parallel sorting is not always faster. For small arrays, the overhead of creating and coordinating parallel tasks may make sequential sorting more efficient.

**Takeaway**

Let Java manage the parallel sorting process.

## 2. Arrays.stream().parallel().sorted() — Declarative parallelism with the Stream API

```
int[] sortedArr2 = Arrays.stream(arr2)
        .parallel()
        .sorted()
        .toArray();
```

**What it does**

- Creates an IntStream from the array.
- Changes the stream's execution mode to parallel.
- Sorts the stream's elements.
- Collects the sorted values into a new array.

**How it works**

- Divides the stream data into portions that can be processed concurrently.
- Uses Fork/Join tasks, normally through the common Fork/Join pool.
- Performs the work as a pipeline of stream operations.
- Preserves primitive int values through IntStream, avoiding ordinary Integer boxing.

**Key idea: Declarative or functional-style parallelism**

The programmer describes the desired operation, while Java determines how the stream pipeline should be divided and executed.

**Important nuance**

sorted() is a **stateful intermediate operation**. It must examine and coordinate information from across the stream before producing the final sorted result. This coordination may reduce the benefits of parallel execution.

**Why it may be slower than parallelSort()**

- Stream creation and pipeline-management overhead.
- Creation of a new output array.
- Coordination required by the stateful sorted() operation.
- A general-purpose stream pipeline may be less specialized than Arrays.parallelSort() for array sorting.

**Takeaway**

Express parallel sorting as part of a declarative processing pipeline.

## 3. ForkJoinPool with ParallelMergeSort — Explicit task decomposition

```
pool.invoke(
   new ParallelMergeSort(arr3, 0, arr3.length - 1)
);
```

**What it does**

- Uses a programmer-defined parallel merge-sort task.

- Recursively divides the array into smaller ranges.
- Sorts independent ranges concurrently.
- Merges the sorted ranges.

**How it likely works**

The ParallelMergeSort task typically:

1. Checks whether the current range is below a specified threshold.
2. Sorts small ranges sequentially.
3. Divides larger ranges into left and right halves.
4. Creates tasks for the two halves.
5. Executes those tasks using the Fork/Join pool.
6. Merges the two sorted halves.

It may use:

- RecursiveAction
- fork() and join()
- invokeAll(leftTask, rightTask)

**Key idea: Task parallelism**

The recursive sorting problem is represented as a hierarchy of tasks. Independent subtasks can execute concurrently.

This example also involves data decomposition because each task works on a different portion of the array. Data parallelism and task parallelism are therefore not mutually exclusive.

**What the programmer controls**

- How the problem is divided.
- The sequential cutoff threshold.
- The task structure.
- The merge implementation.
- Which ForkJoinPool executes the tasks.

The ForkJoinPool, rather than the programmer, schedules the tasks onto worker threads.

**Why it is educationally important**

- Makes task creation and recursive decomposition visible.
- Demonstrates how divide-and-conquer algorithms map to Fork/Join tasks.
- Shows the purpose of thresholds and sequential base cases.
- Illustrates the costs of task creation, coordination, and merging.

**Downsides**

- Requires substantially more code.
- Poor thresholds may create too many small tasks.
- Merging can require additional memory and synchronization.

- An incorrect implementation may introduce race conditions or poor performance.
- A custom implementation may be slower than Java's optimized library methods.

**Takeaway**

You define the task decomposition; the Fork/Join pool schedules and executes the tasks.

| Approach | Primary concept | Programmer responsibility |
|---|---|---|
| Arrays.parallelSort() | Built-in data parallelism | Call the optimized library method |
| Parallel IntStream | Declarative pipeline parallelism | Define the stream operations |
| Custom ParallelMergeSort | Explicit task parallelism | Define tasks, decomposition, thresholds, and merging |